

Technical report: Distilling Context into Parameters for Time-Series Foundation Models for Transportation Forecasting

This report serves as the main technical report for the project *Distilling Context into Parameters for Time-Series Foundation Models for Transportation Forecasting* authored by Felix Thomsen (s221710), Christian Rand (s224930), Alexander Schiøtz (s221221), Bertram Hage (s224918) as part of the DTU course 42578 Advanced Business Analytics 2026.

The following report outlines the complete methodology and framework design and reports the full results. For code implementation we refer to the separate notebook [code_implementation.ipynb](#). For post-hoc analysis code we refer to the files under `post_hoc/`.

Table of contents:

- Introduction
- Background and previous work
- Preliminaries
- Methodology and data
- Experimental setup
- Results and discussion
- Limitations
- Conclusion
- References

Introduction

Modern urban planning, mobility services rely on accurate, real-time forecasting to navigate an increasingly unpredictable world. Deploying state-of-the-art predictive intelligence typically requires massive, centralized computational infrastructure. However, real-world operations frequently demand the opposite: rapid, low-latency decision-making in resource-constrained environments, such as edge computing nodes at traffic intersections or localized fleet management hubs.

This project addresses these operational bottlenecks of state-of-the-art foundational prediction models by proposing a framework that compresses extensive historical context into lightweight, modular parameters. By distilling this long-term data offline, the resulting model can execute highly accurate, real-time forecasts using minimal computational resources. Ultimately, this creates a resilient forecasting pipeline that is economical to deploy at edge nodes and capable of adapting zero-shot to new cities or previously unseen sensors. This ensures uninterrupted analytical support and robust decision-making, even when local data is severely constrained.

Background and previous work

The ability to accurately predict traffic flow is essential for building resilient urban mobility systems capable of adapting to sudden disruptions and maintaining efficiency under stress. In the paper "Time series foundation models as strong baselines in transportation forecasting: A large-scale benchmark analysis" J. Pulido and F. Rodrigues shows that the foundational transformer-based timeseries model Chronos-2 by Amazon achieves state-of-the-art forecasting accuracy in a zero-shot capacity accross various datasets in the traffic domain [1]. While this is promising for the adaptation of foundational models application in traffic speed predictions, a significant limitation of these models is the need to pass long context windows, which forces the architecture to redundantly re-learn the inherent traffic patterns at each inference step. Since latency and memory for transformer models

scales with a time complexity of $O(n^2)$ with context length, this increases the hardware requirements for deployments of such models in a real-world setting limiting their usability.

In "Doc-to-lora: Learning to instantly internalize contexts" R. Charakorn, E. Cetin, S. Uesaka, and R. T. Lange propose an approach for distilling long textual contexts to LoRA adapters for a foundational large language model (LLM) using a learned hypernetwork, in a single forward pass [2]. The hypernetwork is trained to minimize the KL-divergence between the teacher LLMs response with the full context and LoRA-adapted student LLMs response, receiving only a short query. On a needle-in-a-haystack task, the approach successfully maps contexts to adapters, achieving near-perfect zero-shot accuracy on sequences over 4 times the LLM's native window.

Preliminaries

This section briefly outlines preliminary theory used in this project.

LoRA adapters

LoRA (Low-Rank Adaptation) adapters are small, trainable modules added to a pre-trained neural network to adapt it to a new task without updating most of the original weights [3]. Instead of fine-tuning full weight matrices, LoRA learns a low-rank update that approximates changes in weights.

$$\Delta W = BA,$$

with $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $r \ll \min(d_{\text{in}}, d_{\text{out}})$.

This makes training much cheaper and lets you swap or combine task-specific adapters while keeping the same frozen base model.

Chronos-2 foundational model

Chronos-2 is a pre-trained encoder-only Transformer model for time-series forecasting developed by Amazon [4]. It first splits the input time series into fixed-length patches (segments), embeds them, and processes the resulting patch sequence with a stack of attention blocks.

Internally, the architecture alternates between (i) time attention, which applies self-attention along the temporal axis (using rotary position embeddings, RoPE), and (ii) group attention, which aggregates information across related time series within the same group (defined by group IDs) at a given patch index. In our project we do not utilize group attention, and it is a topic for future research how to integrate group attention with our LoRA-based context distillation.

Finally, Chronos-2 produces probabilistic multi-step forecasts with a quantile head that predicts a grid of quantiles for each forecast horizon step.

Methology and data

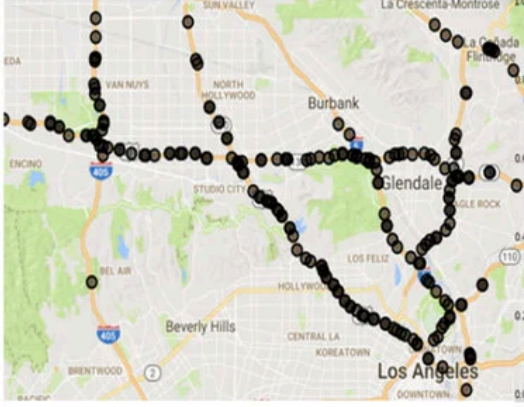
Inspired by the approach by R. Charakorn and colleagues we in this paper transfer the idea to a timeseries prediction task. Specifically, we investigate the application of the approach on the PEMS-BAY dataset, with data from San Francisco's highway system, consisting of cleaned data from 325 under-road sensors used to train AI models to anticipate real-world traffic jams.

Data

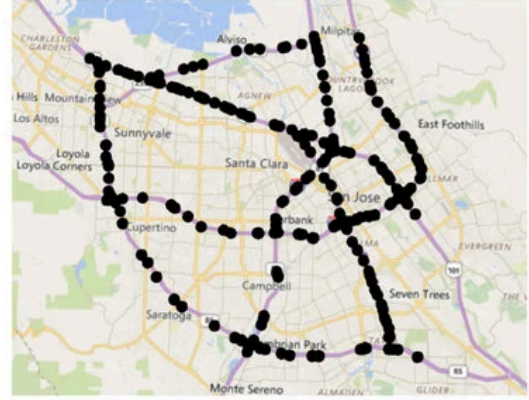
The dataset utilized for training the hypernetwork and evaluating the distilled adapters is the PEMS-BAY traffic speed dataset [5]. Provided by the California Department of Transportation, this dataset features traffic speed readings collected from 325 distinct sensors situated throughout the San Francisco Bay Area. The data spans a timeframe from January 1st 2017 to June 30th 2017, and is recorded at a 5-minute sampling granularity. This high-

frequency structure provides a rigorous testbed for time-series foundation models, as it requires the model to internalize both repetitive weekly patterns and hard to predict localized disruptions.

To evaluate the hypernetwork's transfer-learning capabilities, we additionally utilize the METR-LA dataset for evaluation [6]. It tracks traffic speeds across 207 sensors in Los Angeles over a four-month period (March to June 2012) at a 5-minute granularity, offering a close distributional match to PEMS-BAY in a different geographic setting.



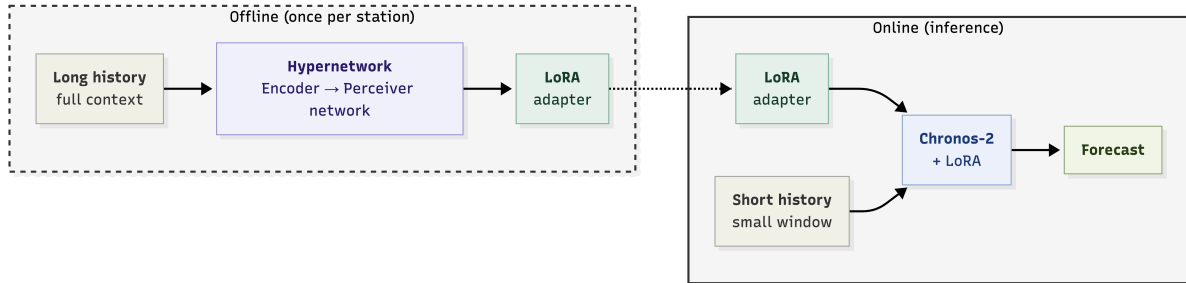
(a) METR-LA



(b) PEMS-BAY

High level design

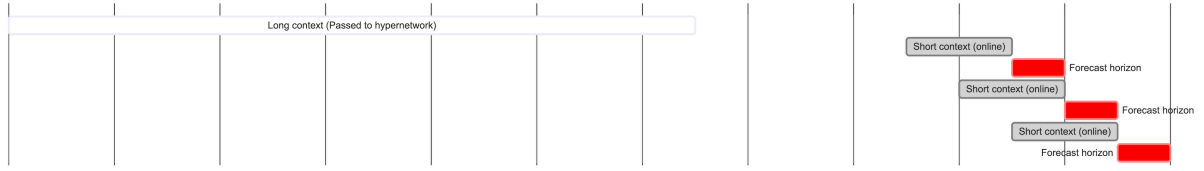
The goal is to have a hypernetwork that, once trained, is able to accept varied length context of a single station and output a useful LoRA adapter. This LoRA adapter can then be applied to the foundational Chronos-2 model and together with a short context window accurately predict traffic speeds at a horizon of 60 minutes. Once the LoRA adapter has been outputted by the hypernetwork for a station it can be used for various short context time windows where the time window appears after the long context.



Formally, let $\mathcal{D}$ be a multivariate time-series dataset comprising N stations observed over T timesteps. For a given station $i \in \{1, \dots, N\}$ and a forecast origin t , the task is to predict the sequence of future values over a forecast horizon H , corresponding to the time indices $\{t + 1, t + 2, \dots, t + H\}$.

To generate this prediction, the proposed architecture relies on two distinct observation windows:

- Short Context (C_{short}): A recent historical window of length W_{short} that immediately precedes the forecast origin. It is defined as the set of indices $\{t - W_{short} + 1, \dots, t\}$. This serves as the direct input to the LoRA-adapted foundation model during online inference.
- Long Context (C_{long}): An extended historical window of length W_{long} utilized offline by the hypernetwork to distill the station's temporal dynamics into adapter parameters. This window spans a set of indices $\{t_{start}, \dots, t_{end}\}$ such that $t_{end} < t - W_{short} + 1$. Thus, the short context may immediately precede the long context or there may be a time gap in between. In our implementation C_{long} is fixed for all forecast horizons t .



Hypernetwork design

The goal of the hypernetwork is to compile a long historical window for a single station into a small set of LoRA parameters that can later be reused for many short-context forecasts. Concretely, we learn a mapping

$$h_\phi : C_{\text{long}} \mapsto \{\Delta W^{(\ell,m)}\}_{\ell=1..12, m \in \{q,k,v,o\}},$$

where each $\Delta W^{(\ell,m)}$ is represented by a low-rank factorization (LoRA) for Chronos-2's time self-attention projections: query (q), key (k), value (v), and output (o) in each of the 12 encoder blocks.

Context encoder

We first embed the long context with a frozen encoder. For this we use the first 8 layers of the Chronos-2 model. Given the raw time-series values of shape $[1, W_{\text{long}}]$, the context encoder returns the last hidden states

$$Z \in \mathbb{R}^{1 \times S \times 768},$$

where S is the number of context patches produced by Chronos-2. Keeping this encoder reduces the number of parameters needed to be learned by the hypernetwork, and we assume the initial layers of the Chronos-2 model already has learned a valuable representation of time-series data.

Perceiver aggregator

The number of context patches S can vary, so we need a module that turns a variable-length sequence Z into a fixed-size representation. Inspired by Doc-to-LoRA, we use a Perceiver-style aggregator: a small set of learned latent query-vectors cross-attend to Z and collect the relevant information. We use 32 latent queries in our implementation.

The Perceiver outputs a fixed set of $N_{\text{out}} = 384$ vectors, computed as $12 \text{ layers} \times 4 \text{ modules} \times \text{LoRA rank} = 8$, each of size 128. This can be thought of as one vector per layer, per module, per rank-slot.

Projection to LoRA weights

Finally, each 128-dimensional vector is passed through a 1-layer residual MLP and then through a layer/module-specific linear head that outputs the two LoRA matrices for each layer ℓ and module m

$$A^{(\ell,m)} \in \mathbb{R}^{r \times d_{\text{model}}}, \quad B^{(\ell,m)} \in \mathbb{R}^{d_{\text{model}} \times r},$$

with $r = 8$ and $d_{\text{model}} = 768$.

Training setup

The training objective is to learn a hypernetwork h_ϕ that maps a long context window C_{long} for a single station to LoRA weights for Chronos-2. During training, the frozen Chronos-2 backbone is never updated, only the hypernetwork parameters are optimized.

The training setup follows the high-level structure:

1. Sample a batch of long-context windows. A sample corresponds to a specific station and a start index of a rolling window in time.
2. For each long context, we select Q forecast origins spaced by a fixed stride within the admissible range. This yields Q short-context windows $\{C_{\text{short}}^{(q)}\}_{q=1..Q}$ and associated targets, all sharing the same long context. The

hypernetwork must therefore produce an adapter that performs well across several forecast origins, rather than overfitting to a single moment. This is analogous to the approach in Doc-to-LoRA where multiple text-queries were generated for a single piece of long context.

3. For each sample, encode long context and generate LoRA weights according to the hypernetwork design described above.
4. The generated LoRA weights are injected into Chronos-2, and the LoRA-adapted model is run on each short-context query to produce probabilistic forecasts (quantile predictions) for the horizon H .
5. The loss is averaged over the batch, the Q queries per long context, and the H forecast steps, and gradients flow only back into the hypernetwork.

For the training of the hypernetwork we experiment with two training objectives.

Training with distillation targets

Here we define:

- **Teacher:** frozen Chronos-2 with the full available C_{long} followed by C_{short} passed.
- **Student:** frozen Chronos-2 augmented with the LoRA adapter produced from the hypernetwork and C_{long} , run only on the short-context C_{short} .

Let the teacher and student quantile outputs

$$\hat{Q}^{(T)} \in \mathbb{R}^{Q \times K \times H}, \quad \hat{Q}^{(S)} \in \mathbb{R}^{Q \times K \times H},$$

where K is the number of quantile levels and H is the prediction length, we minimize:

$$\mathcal{L}_{\text{teacher}} = \frac{1}{QKH} \sum_{q=1}^Q \sum_{k=1}^K \sum_{h=1}^H \text{SmoothL1}(\hat{Q}_{q,k,h}^{(S)} - \hat{Q}_{q,k,h}^{(T)}),$$

where

$$\text{SmoothL1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}.$$

This objective encourages the LoRA-adapted student to match the full-context probabilistic forecast of the teacher.

Training with ground truth targets

Here we remove the teacher entirely and supervise the student directly on the ground truth future values $y \in \mathbb{R}^{Q \times H}$. Since Chronos-2 predicts quantiles, we use a loss computed from quantile losses.

For a quantile level $\tau \in (0, 1)$ and error $u = y - \hat{q}_\tau$, the quantile loss is

$$\rho_\tau(u) = \max(\tau u, (\tau - 1)u).$$

With quantile levels $\{\tau_k\}_{k=1..K}$, we compute the combined loss as the average quantile loss across quantiles:

$$\mathcal{L}_{\text{gt}} = \frac{2}{QKH} \sum_{q=1}^Q \sum_{k=1}^K \sum_{h=1}^H \rho_{\tau_k}(y_{q,h} - \hat{Q}_{q,k,h}^{(S)}).$$

By scaling the loss with 2 the interpretation becomes close to that of the mean-absolute-error.

This objective directly optimizes forecasting quality against the data, while still training only the hypernetwork parameters.

Hierarchical length jitter

To increase the generalization of our hypernetwork we add Gaussian noise to the length of the long and short context windows during training. This is done in a hierarchical manner to keep batches memory efficient.

For each batch we first sample a batch-level mean length from a Gaussian distribution, then sample per-sample lengths around that mean, again from a Gaussian distribution.

During initial validation runs this greatly increased the generalizability of the hypernetwork and the downstream evaluation performance.

Experimental setup

The following details the concrete experimental settings used for training and evaluation during experiments.

Dataset split

Training covers 2017-01-01 to 2017-04-01 (~3 months). Validation covers 2017-04-01 to 2017-05-01 (~1 month). The test set is reserved as the final 20% of the temporal range corresponding to the last 36 days, 2017-05-26 to 2017-06-30.

Long context windows

We run two settings for the long context going to the hypernetwork: 2016 time-steps (1 week) and 4032 time-steps (2 weeks). The student Chronos-2 model receives a short context of 288 time-steps (1 day). The forecast horizon is 12 steps (1 hour), evaluated at 15-, 30-, and 60-minute horizons.

Hierarchical length jitter

During training, context lengths are sampled per batch using hierarchical Gaussian jitter to prevent the hypernetwork from overfitting to fixed-length inputs. A batch-mean long-context length is drawn with $\sigma_{\text{outer}} = 320$ steps, then per-sample lengths are drawn with $\sigma_{\text{inner}} = 96$ steps. Short-context lengths use $\sigma_{\text{outer}} = 40$, $\sigma_{\text{inner}} = 16$. Lengths are clamped to $[1008, 4032]$ (long) and $[144, 432]$ (short).

Multi-query training

Each long context is paired with $n = 16$ forecast origins, spaced 48 steps (4 hours) apart. The hypernetwork is optimized against the aggregate loss across all 16 queries per context, encouraging context-level generalization rather than point-specific overfitting.

Target modes

Two supervision objectives are evaluated: training with distillation targets and ground truth targets. These supervision objectives and their respective loss is described in detail in the methodology section. To avoid re-computing teacher quantiles each epoch for the distillation target objective we build a cache ahead of the training loop containing the teacher targets.

Optimizer and learning rate

We use the AdamW optimizer with $\text{lr} = 4 \times 10^{-5}$ and weight decay 0.01. A linear warmup (start factor 10^{-2} , 100 steps) is followed by cosine annealing to $\eta_{\text{min}} = 10^{-7}$. Gradient clipping is applied at norm 1.0.

Batch and gradient accumulation

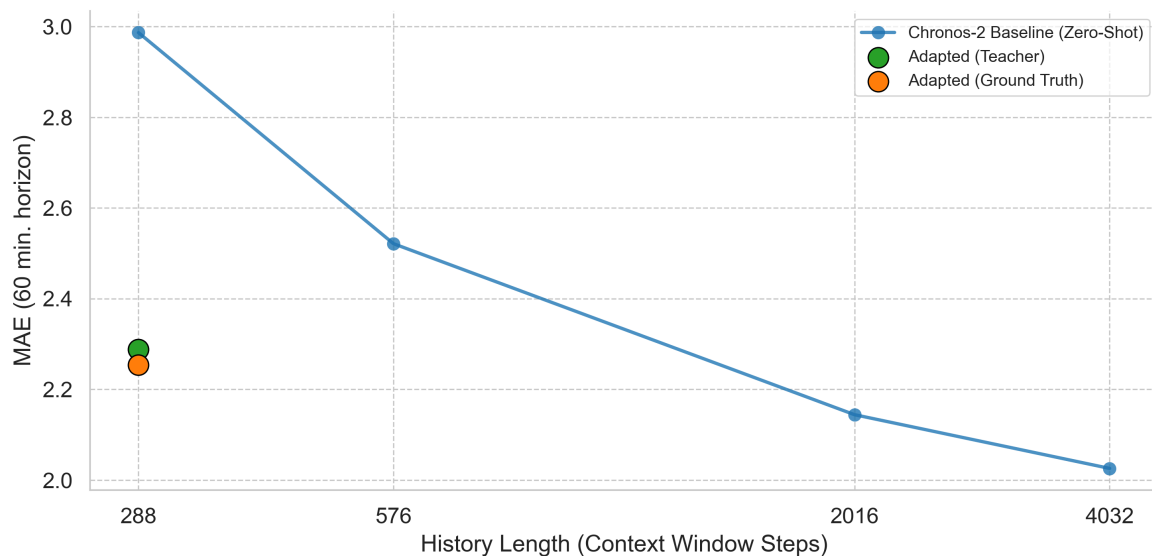
We use a batch size is 64, with 2 gradient accumulation steps, yielding an effective batch size of 128. While VRAM requirements for the training run is high we noticed that a high effective batch size was necessary to stabilize training. Training runs for up to 100 epochs with early stopping (patience = 10 epochs on validation loss). A seed of 42 is used for all random operations.

GPU hardware and tracking

All experiments run on a single NVIDIA Tesla A100 PCIE 80 GB with 4 CPU cores (16 GB RAM), using PyTorch CUDA on the DTU HPC cluster. Runs are logged to Weights & Biases and checkpoints are saved to disk for downstream evaluation.

Results

The figure below shows the average mean-absolute-error (MAE) at the 60 minutes horizon plotted against the context lengths. Adapted models are using 2016 timesteps (7 days) for the hypernetwork to generate their LoRA and both the network trained with the teacher and ground truth supervision are shown. The baseline represents the vanilla Chronos-2 model performance without any adaptation at various context lengths.



For the baseline we see performance increase with context length, albeit with diminishing returns for longer context lengths. Our adapted model breaks this curve achieving a better MAE with half the context of the baseline at 576 steps (2 days), and getting close to the performance of the baseline at 2016 steps (7 days). While the adapted model with the ground truth supervision outperforms the adapted model with the teacher supervision this is not a stark difference. The primary objective of this project was to determine whether a long historical context window could be effectively compressed into lightweight model parameters for time-series forecasting. Based on the evaluation of the average mean-absolute-error (MAE) at a 60-minute forecast horizon, the results strongly validate our approach.

Model	h15 MAE	h15 Coverage	h30 MAE	h30 Coverage	h60 MAE	h60 Coverage
Baseline 288 steps	1.766	0.758	2.266	0.758	2.987	0.755
Baseline 576 steps	1.65	0.753	2.04	0.755	2.521	0.756
Baseline 2016 steps	1.542	0.758	1.835	0.765	2.144	0.771
Baseline 4032 steps	1.506	0.754	1.769	0.763	2.026	0.771
Adapted 2016 steps (Teacher)	1.563	0.746	1.905	0.748	2.288	0.753
Adapted 2016 steps (Ground Truth)	1.562	0.802	1.893	0.791	2.254	0.784
Adapted 4032 steps (Teacher)	1.571	0.734	1.919	0.738	2.296	0.747
Adapted 4032 steps (Ground Truth)	1.563	0.809	1.893	0.811	2.266	0.812

The above table shows the full results for both MAE and coverage at 15 minutes (h15), 30 minutes (h30) and 60 minutes (h60) forecast horizon. While MAE can be thought of as prediction error, coverage measures the reliability of the model's uncertainty estimation by quantifying the percentage of time the ground-truth observations fall within the predicted quantile intervals.

For the baseline the MAE performance shows a similar picture for h15 and h30 as for h60 where performance increase with context length but with diminishing returns. For the baseline, the effect of context length is less pronounced for Coverage for h15 and h30.

For the adapted models we see a general trend of the ground truth supervision outperforming teacher supervision and that longer long context lengths increases performance. The differences, however, are not great, and especially the long context lengths appears to have very minimal impact on performance suggesting a plateau for performance around the long context steps 2016 steps.

It can come as a surprise that the models adapted with the ground truth supervised hypernetworks are not outperforming the teacher-supervised models more significantly since the training objective is less biased. However, this aligns with the findings of J. Pulido and F. Rodrigues [1], who demonstrated that Chronos-2 achieves state-of-the-art zero-shot accuracy, often surpassing custom-tailored architectures. This convergence suggests we are operating at the theoretical boundary of what can be achieved given enough historical context. The hypernetwork's primary value is thus reaching this high-performance ceiling while utilizing a fraction of the inference-time context and memory.

Transfer learning

To assess and stress-test the generalization of the hypernetwork we experimented with training it on only a small subset of stations and evaluating on the held out stations.

We experimented with training the hypernetwork on only 20% of the stations and evaluating on the remaining 80% of stations unseen to the hypernetwork during training.

Model	h15 MAE	h15 Coverage	h30 MAE	h30 Coverage	h60 MAE	h60 Coverage
Holdout 2016 steps (Teacher)	1.569	0.744	1.925	0.741	2.327	0.747
Baseline 2d	1.65	0.753	2.04	0.755	2.521	0.756
Baseline 7d	1.542	0.758	1.835	0.765	2.144	0.771
Adapted 2016 steps (Teacher)	1.563	0.746	1.905	0.748	2.288	0.753
Adapted 2016 steps (Ground Truth)	1.562	0.802	1.893	0.791	2.254	0.784

Judging from the results, the hypernetwork trained with this holdout approach achieved nearly similar results to the one trained over all stations. This suggests that the hypernetwork achieves a high degree of generalization enabling transfer learning.

Similarly, to further stress-test the generalization, we tried taking the hypernetwork trained on the PEMS-BAY dataset and used it to generate LoRA adapters for stations in the METR-LA dataset, which is a completely different city.

Model	h15 MAE	h15 Coverage	h30 MAE	h30 Coverage	h60 MAE	h60 Coverage
Baseline 288 steps	3.594	0.766	4.456	0.751	5.807	0.718
Baseline 576 steps	3.572	0.757	4.412	0.745	5.6	0.715
Baseline 2016 steps	3.372	0.746	4.058	0.743	4.895	0.712
Adapted (PEMS-BAY Hypernet) 2016 steps (Teacher)	3.415	0.758	4.142	0.749	5.136	0.724

Interestingly, also here we see good performance on the METR-LA even though the hypernetwork has never seen the data from the city during training. This demonstrates that the hypernetwork does not merely memorize dataset-specific structures, but instead captures a fundamental, location-agnostic mapping for compressing temporal patterns into adapter weights. Consequently, the generated LoRAs can effectively bridge the performance gap between short and long contexts, even when deployed zero-shot across entirely different road networks.

Limitations

While the proposed framework successfully compresses historical context into adapter weights, several limitations must be acknowledged regarding its design:

First of all, there is a risk the hypernetwork might be learning a hidden way of encoding the 24-hour seasonal patterns into the LoRA. From our results there is no guarantee this approach would work on non-seasonal data, such as retail demand for new products or financial events, where history is less predictable.

Secondly, our approach used only raw timeseries data. Chronos-2 is able to encode various other non-timeseries data and traffic is rarely univariate. Due to limited time we were not able to investigate the ability to encode non-timeseries data using our framework. The Doc-to-LoRA project by R. Charakorn et al. showed that by using the vision part of a vision language model as encoder for the hypernetwork a LLM without vision were able to ingest non-text context via LoRA adapters. If this finding carries over to the timeseries domain this could be a significant further contribution to our proposed framework.

Furthermore, while this project focus on computationally cheap inference, training requires a large VRAM footprint. This could become a bottleneck in the adaptation of this approach, although technically the hypernetwork would only need to be trained once.

Lastly, since training the hypernetwork takes significant time and memory, we were only able to experiment with a small set of hyperparameters. Thus, the framework might be underperforming compared to the reported results, as a more exhaustive tuning phase could potentially unlock better context compression and predictive accuracy.

Conclusion

This project successfully demonstrated the viability of compressing long-term time-series history into lightweight model parameters. By adapting the Doc-to-LoRA framework for the Chronos-2 foundation model, we reduced the required inference context window by nearly 85% (from 2016 steps down to 288 steps) while retaining a competitive forecasting accuracy.

Consistent with recent large-scale benchmarks, Chronos-2 already exhibits strong zero-shot capabilities. Our findings indicate that our hypernetwork framework effectively unlocks it for resource-constrained environments. The performance convergence between our teacher-distilled and ground-truth supervised models suggests that we have captured the maximum exploitable signal from the historical context. Our primary contribution, thus, is not inventing a more accurate forecast, but engineering a dramatically more efficient and deployable one.

Furthermore, our transfer learning experiments address the challenge of operating with limited data. The hypernetwork demonstrated an ability to generate effective adapters for entirely unseen stations—and even entirely different cities zero-shot. In a real-world setting, this provides urban planners and business analysts with a highly resilient tool capable of adapting immediately to urban disruptions, newly deployed sensor networks, or sudden changes in traffic topology where long-term historical data simply does not yet exist.

Ultimately, this framework bridges the gap between the heavy computational demands of foundational models and the agile, resilient needs of real-world business analytics, proving that we can internalize the past to better and more efficiently navigate the uncertainties of the future.

References

- [1] J. Pulido and F. Rodrigues, "Time series foundation models as strong baselines in transportation forecasting: A large-scale benchmark analysis," *arXiv preprint arXiv:2602.24238*, 2026.
- [2] R. Charakorn, E. Cetin, S. Uesaka, and R. T. Lange, "Doc-to-lora: Learning to instantly internalize contexts," *arXiv preprint arXiv:2602.15902*, 2026
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *CoRR*, vol.abs/2106.09685, 2021
- [4] A. F. Ansari, O. Shchur, J. K\"{u}ken, A. Auer, B. Han, P. Mercado, S. S. Rangapuram, H. Shen, L. Stella, X. Zhang, M. Goswami, S. Kapoor, D. C. Maddix, P. Gueron, T. Hu, J. Yin, N. Erickson, P. M. Desai, H. Wang, H. Rangwala, G. Karypis, Y. Wang, and M. Bohlke-Schneider, "Chronos-2: From univariate to universal forecasting," 2025
- [5] S. Sun, "PEMS-BAY," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/scchuy/pemsbay> [6] HuuAnnnn, "METR-LA," Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/annnnnguyen/metr-la-dataset>